GENERATIVE AI

Lab Experiment – 01:

1. **Develop a predictive text system using N-Gram and HMM (Hidden Markov Models) to predict the next word based on previous context. Dataset: Brown Corpus or Wikipedia dataset.**

Code:

```
!pip install nltk

import nltk
from nltk.corpus import brown
from nltk.util import ngrams
from collections import defaultdict, Counter
nltk.download('brown')
nltk.download('universal_tagset')
sentences = brown.sents()
sentences = [[word.lower() for word in sent] for sent in sentences]
words = [word for sent in sentences for word in sent]
print("Sample sentence:", sentences[0])
```

N-GRAM MODEL:

```
bigrams = list(ngrams(words, 2))
bigram_model = defaultdict(Counter)

for w1, w2 in bigrams:
    bigram_model[w1][w2] += 1
def predict_next_word_ngram(word, top_n=5):

    if word not in bigram_model:
        return "Word not found"

    predictions = bigram_model[word].most_common(top_n)

    return [w for w, count in predictions]
predict_next_word_ngram("the")
```

OUTPUT:

```
['first', 'same', 'most', 'other', '``']
```

Hidden Markov Model (HMM):

```
tagged_sentences = brown.tagged_sents(tagset='universal')

transition_counts = defaultdict(Counter)
emission_counts = defaultdict(Counter)

for sentence in tagged_sentences:

    prev_tag = None

    for word, tag in sentence:

        word = word.lower()

        emission_counts[tag][word] += 1

        if prev_tag is not None:
            transition_counts[prev_tag][tag] += 1

        prev_tag = tag

transition_counts = defaultdict(Counter)
emission_counts = defaultdict(Counter)

for sentence in tagged_sentences:

    prev_tag = None

    for word, tag in sentence:

        word = word.lower()

        emission_counts[tag][word] += 1

        if prev_tag is not None:
            transition_counts[prev_tag][tag] += 1

        prev_tag = tag

predict_next_word_hmm("the")
```

OUTPUT:

```
['time', ',', '.', 'man', 'is']
```